

RAG Interview Prep Guide

Question Bank + System-Design Playbook
AI Engineer / RAG Architect

RAG Interview Question Bank

70+ questions with model answers, grouped to match the roadmap and tagged by level:

[JUNIOR] Junior · [MID] Mid · [ARCH] Senior/Architect. Read the model answer, then close the doc and say it out loud in your own words.

§1 — Fundamentals

[JUNIOR] What is RAG?

Retrieval-Augmented Generation: before the LLM answers, you retrieve relevant text from your own data and put it in the prompt, so the model answers from supplied facts rather than only its training memory. It's three layers — index, retrieve, generate.

[JUNIOR] What problems does RAG solve that a raw LLM has?

Knowledge cutoff (it can use fresh/private data), hallucination (answers are grounded in retrieved context), and lack of provenance (you get citations).

[MID] RAG vs fine-tuning vs long context — when each?

Fine-tuning teaches *behavior/style/format* and is costly to update; long context pastes everything (simple but expensive, limited, and suffers "lost in the middle"); RAG injects *fresh, private facts* at query time with citations and is cheap to update. They're complementary: fine-tune for how to act, RAG for what's true now.

[MID] Why is "retrieval quality decided during indexing"?

If the right information isn't chunked and embedded into the index in a findable form, no prompt or model can recover it at query time. ~80% of RAG failures originate in loading/chunking.

[ARCH] When would you NOT use RAG?

When the knowledge is general (no private/fresh data needed), when the corpus is tiny enough to fit in context cheaply, when you need behavioral change rather than facts (fine-tune), or when latency budgets can't absorb a retrieval hop. Don't add RAG reflexively.

[ARCH] Walk me through what happens on a single query, end to end.

Authn/z input guardrail (router) embed query hybrid retrieve (vector + BM25 RRF) rerank assemble grounded prompt with citations generate (stream) output guardrail/faithfulness check log/trace respond. Indexing happens offline separately.

§2 — Embeddings & Vectors

[JUNIOR] What is an embedding?

A fixed-length vector representing the meaning of text such that semantically similar text is geometrically close. We retrieve by nearest-neighbor search over these vectors.

[JUNIOR] Cosine vs dot product vs Euclidean?

Cosine = angle (direction) similarity, the text default; dot product equals cosine for normalized vectors; Euclidean is straight-line distance (smaller = closer). OpenAI embeddings are normalized, so cosine and dot rank identically.

[MID] Why must index and query use the same embedding model?

Different models produce vectors in different, non-comparable spaces. Mixing them makes similarity meaningless.

[MID] Bigger embedding dimension — always better?

No. Higher dimensions can improve accuracy but cost more storage and slightly slower search; gains plateau. Some models support Matryoshka truncation to trade accuracy for size. Choose by evaluation, not by dimension count.

[ARCH] Your retrieval is weak on a specialized domain (legal/medical). What do you try?

Domain-specific or fine-tuned embeddings, hybrid search to catch exact jargon/codes, better chunking, query transformations (HyDE), and reranking. Measure each change on a golden set; embeddings can yield ~20% swings but chunking often more.

§3 — Chunking

[JUNIOR] Why chunk documents?

A whole document embeds to a meaningless average and won't fit cleanly in a prompt. Chunks are the unit of retrieval — small enough to match precisely, large enough to be meaningful.

[MID] Name chunking strategies and when each fits.

Fixed-size (simple); fixed+overlap (sane default, preserves boundary facts); recursive (respects paragraphs/sentences); structure-aware (Markdown/HTML/code); semantic (splits on topic shift); small-to-big/parent (match small, return big). Prose recursive ~512 tokens/15% overlap; structured split on headings; code by function.

[MID] Good default chunk size?

~512 tokens with 10–20% overlap is a strong 2026 baseline, but it's a starting point — sweep 256/512/1024 and pick by RAGAS context precision/recall on your corpus.

[ARCH] A fact keeps getting missed in retrieval. Diagnose.

Check if the fact survived chunking intact (boundary split?), if it's findable (needs keyword/hybrid for a code/name?), if top-k is too low, and if the embedding is blurry (chunk too big/multi-topic). Most often it's a chunking or hybrid issue, not the model.

[ARCH] What is contextual retrieval and what does it cost?

Prepend an LLM-generated sentence situating each chunk in its document before embedding, which sharply improves recall for context-dependent chunks. Cost: one LLM call per chunk at index time (cache it).

§4 — Vector Databases

[JUNIOR] What does a vector DB store and do?

The vector, the original text, and metadata; it serves fast approximate nearest-neighbor search plus metadata filtering.

[MID] HNSW vs IVF?

HNSW is a navigable-graph index — fast, high recall, handles updates; the modern default. IVF clusters vectors and searches nearby clusters — memory-efficient at huge scale; with PQ it compresses vectors. Trade-off is recall vs latency vs memory (tune `ef_search` / `nprobe`).

[MID] How do you choose a vector DB?

Fit it to your stack/ops: Chroma for prototyping, pgvector if you're on Postgres, Qdrant/Weaviate for dedicated self-hosted with strong hybrid, Pinecone to avoid ops, Milvus/Vespa at billions of vectors. The DB is rarely the bottleneck — retrieval quality and ops are.

[ARCH] How do you enforce per-user access control in RAG?

Store permission metadata (`tenant_id`/roles) on every chunk and apply it as a pre-filter so search can only ever return chunks the user is allowed to see. Never rely on the prompt to hide data.

[ARCH] How do you keep the index fresh?

Event/cron-driven indexing pipeline separate from the query path; idempotent upserts with stable chunk IDs; delete stale chunks; incremental over full re-index; define and meet a freshness SLA.

§5 — Retrieval

[JUNIOR] What is top-k?

The number of chunks retrieved per query. Too few misses info; too many adds noise/cost. Typical: retrieve ~20, rerank to 3–5.

[MID] Dense vs sparse retrieval?

Dense (vector) captures meaning/paraphrase but can miss exact terms; sparse (BM25) nails exact codes/names/jargon but misses synonyms. Opposite weaknesses combine them.

[MID] What is hybrid search and why RRF?

Run vector + BM25 in parallel and fuse. RRF (Reciprocal Rank Fusion) merges by rank — $1/(k+\text{rank})$ — so it ignores incompatible score scales. Hybrid is the 2026 default; ~1–9% recall gains, more on code/name-heavy corpora.

[MID] Bi-encoder vs cross-encoder / what is reranking?

A bi-encoder encodes query and doc separately (fast, first-pass over millions). A cross-encoder reads query+doc together (accurate, slow). Reranking retrieves wide and cheap (top-20) then re-scores narrow and accurate (to top-4). It's one of the cheapest big quality wins.

[ARCH] What is "lost in the middle" and how do you fight it?

LLMs attend best to the start/end of context and overlook the middle. Mitigate by reranking to fewer chunks, ordering most-relevant first (and optionally repeating key context at the end), and preferring high-precision short context over long noisy context.

[ARCH] What is MMR and when do you use it?

Maximal Marginal Relevance trades some relevance for diversity to avoid filling the context with near-duplicate chunks. Use it when top results are redundant.

§6 — Generation & Prompting

[JUNIOR] How do you reduce hallucination via the prompt?

Instruct "answer only from the context," forbid outside knowledge, give an explicit refusal sentence for missing info, require citations, and set temperature 0.

[MID] How do citations work in RAG?

Label each chunk with an index/source in the prompt, instruct the model to cite [n], and render those back to source+page from metadata. Optionally verify cited chunks exist.

[MID] What's a relevance gate?

If the top retrieval/rerank score is below a threshold, skip generation and return a refusal — don't answer from weak context.

[ARCH] Hallucination defense is layered — list the layers.

Fix retrieval first; grounding prompt + refusal path; relevance gate on score; citation enforcement; faithfulness evaluation; optional self-grading (CRAG) before returning. No single layer suffices.

[ARCH] When do you use structured output?

When a downstream system consumes the answer — request JSON (answer, citations, confidence) via the provider's structured-output mode for reliability.

§7 — Evaluation

[JUNIOR] Why evaluate instead of eyeballing?

RAG has many knobs; intuition fools you. Metrics let you change one thing, re-measure, and know if it helped.

[MID] The four core RAGAS metrics?

Faithfulness (answer supported by context), answer relevancy (addresses the question), context precision (retrieved chunks relevant & well-ranked), context recall (all needed info retrieved — needs ground truth).

[MID] How do these localize a failure?

Precision/recall low retrieval problem (chunking, hybrid, top-k). Faithfulness/relevancy low with good retrieval generation problem (prompt/model). This split tells you which half to fix.

[ARCH] How do you build a golden dataset?

Hand-write representative (question, ground_truth) pairs covering easy, multi-hop, ambiguous, and out-of-scope cases; optionally LLM-generate candidates but review them by hand; aim for 30–50+. Include refusal cases.

[ARCH] Offline vs online evaluation?

Offline runs a fixed golden set (great for CI, regression-proofing). Online samples real production traffic and scores it continuously to catch drift offline tests miss. Do both; wire offline eval into CI to block regressions.

[ARCH] LLM-as-judge — risks?

Bias (especially if judge = generator), inconsistency, and cost. Mitigate with a different/stronger judge model, clear rubrics, multiple samples, and human spot-checks.

§8 — Advanced RAG

[MID] What is HyDE?

Have the LLM write a hypothetical answer, embed *that*, and search with it — it's shaped like real documents, fixing query/document asymmetry. Risk: a hallucinated hypothetical can mislead retrieval.

[MID] Multi-query expansion?

Generate several rephrasings of the query, retrieve for each, and fuse — catches synonym/phrasing gaps at the cost of extra calls/searches.

[MID] Parent-document / small-to-big?

Embed small chunks for precise matching but return their larger parent for the LLM to read — precision of small with context of big.

[ARCH] What is contextual compression?

After retrieval, strip each chunk to only the query-relevant sentences (via an LLM/extractor) to cut noise and tokens and fight "lost in the middle."

[ARCH] You're told to "make RAG better." How do you decide what to add?

Establish a golden-set baseline, identify whether retrieval or generation is the bottleneck, then test candidate techniques one at a time, keeping only those that move the target metric enough to justify added latency/cost. Resist adding everything.

§9 — Agentic RAG

[MID] What is agentic RAG?

Retrieval as a decision loop: the LLM decides whether/what to retrieve, evaluates results, retrieves again, calls tools, and self-corrects — vs a fixed retrieve-once pipeline.

[MID] What is Adaptive RAG / routing and why is it valuable?

Classify each query and send it to the cheapest sufficient pipeline (no-retrieval / simple / multi-hop / tool). It matches cost to difficulty — the top cost-control pattern in 2026.

[ARCH] Risks of agentic RAG and mitigations?

Latency and cost explode (multiple LLM calls), non-termination, and error compounding. Mitigate with routing (only hard queries pay), max-step budgets + fallbacks, per-step grading/validation, and full tracing.

[ARCH] When is an agent over-engineering?

When single-shot retrieval answers the vast majority of queries. Start simple; add agentic behavior only where data shows it's needed.

[ARCH] What is CRAG (corrective RAG)?

Grade retrieved context (and/or the draft answer); if it's poor, take corrective action — rewrite the query and re-retrieve, fall back to another source, or refuse.

§10 — Graph RAG

[MID] What is Graph RAG and when does it win?

Build a knowledge graph of entities/relationships and retrieve by traversal. It wins on multi-hop ("who reports to the owner of X?") and global/summarizing questions where vector similarity can't chain or aggregate facts.

[ARCH] How is the graph built and served?

LLM-extract (subject, relation, object) triples graph store (Neo4j); for local queries traverse from entry entities, for global queries summarize graph communities. Usually hybridized with vector search.

[ARCH] Costs/downsides of Graph RAG?

Expensive extraction, entity-resolution complexity, and graph-freshness maintenance. Use it only for relational/multi-hop domains; don't default to it for simple passage lookup.

§11 — Production & Architecture

[MID] Why separate indexing and query pipelines?

They have different scaling and schedules; coupling them means a heavy re-index spikes query latency. Keep ingestion offline/event-driven and the query path always-on and low-latency.

[MID] Biggest cost driver and how to control it?

LLM tokens. Control with tight retrieval (fewer/shorter chunks), model tiering (small model by default), semantic caching, and adaptive routing.

[MID] What is semantic caching?

Embed the query; if a previous query is similar enough, return its answer. Catches paraphrases, not just exact matches — big latency/cost win. Invalidate on document change.

[ARCH] What NFRs do you design to, with numbers?

Latency (TTFT p90 < ~2s, track p50/p90/p99), throughput/scale (stateless autoscaled query service), cost/query budget, reliability (99.9% uptime, retries, graceful degradation), security (per-tenant filters, prompt-injection defense, PII), compliance (residency, right-to-be-forgotten).

[ARCH] How do you defend against prompt injection in retrieved docs?

Treat retrieved text as untrusted data, not instructions; sanitize/strip; instruct the model accordingly; add input/output guardrails; constrain tools and permissions. Assume a document may contain "ignore your instructions."

[ARCH] What does observability for RAG look like?

Per-request tracing of every stage (query, retrieved chunk IDs+scores, reranked set, final prompt, model, tokens, latency, cost), metrics (latency percentiles, cache hit rate, refusal rate, cost/query), and online eval sampling with drift alerts.

[ARCH] How do you support "right to be forgotten"?

Be able to delete a user's documents *and* all derived chunks/embeddings (and purge caches) — stable IDs and metadata make this tractable.

[ARCH] Build vs buy a RAG platform?

Build (frameworks + your code) for control, customization, and cost at scale, owning ops/eval; buy (managed RAG) for speed and low ops at the cost of control and per-query price. Decide on data sensitivity, scale, team size, time-to-market, and customization needs.

§12 — Behavioral / judgment (architect)

[ARCH] Tell me about a RAG quality problem and how you fixed it.

Frame it as: symptom measured with golden set localized (retrieval vs generation) hypothesis one-variable change re-measured shipped + monitored. The *process* is the answer they want.

[ARCH] How do you keep a RAG system from silently regressing?

Golden-set eval in CI with thresholds, online eval with drift alerts, tracing for diagnosis, and a feedback loop (thumbs up/down) feeding the golden set.

[ARCH] A stakeholder wants 100% accuracy. How do you respond?

Reframe to measurable targets and trade-offs: define metrics and an acceptable bar, show the cost/latency curve, add refusal for low-confidence cases, and set up human review for high-stakes queries. Manage expectations with data.

Drill tip: for any [ARCH] answer, be ready with a **trade-off** and a **number**. "It depends" is fine only when followed by *what* it depends on.

RAG System-Design Playbook (Architect Round)

The architect interview is rarely "define RAG." It's "design a RAG system for X under these constraints, and defend every decision." This playbook gives you a repeatable framework, two fully worked scenarios, and a mock interview.

The framework: a 7-step script for any RAG design question

Say these steps out loud — interviewers grade *structure* as much as content.

1. Clarify requirements (don't skip — they're testing this)

Ask about:

- **Corpus:** how many docs, what types (PDF/HTML/tables/code), how often they change, languages.
- **Users & scale:** concurrent users, QPS, peak vs average.
- **Quality bar:** accuracy needs, must it cite sources, must it refuse when unsure.
- **NFRs:** latency target (TTFT/p90), cost ceiling, uptime SLA.
- **Security/compliance:** multi-tenant? PII? data residency? access control?

Restate the requirements before designing.

2. Sketch the high-level architecture

Draw the two pipelines: **indexing** (load clean chunk embed upsert) and **query** (auth guardrail route retrieve rerank generate guardrail respond), plus vector DB, cache, and observability. (See docs/11.)

3. Decide the indexing strategy

Loaders per source; cleaning; **chunking** (recursive ~512/overlap, structure-aware where possible); embedding model; vector DB choice; metadata schema (source, page, tenant, date, permissions); freshness mechanism (event-driven, idempotent upserts, deletes).

4. Decide the retrieval strategy

Hybrid (vector + BM25 + RRF) + **reranking**; top-k (retrieve ~20 rerank ~4); metadata filters (incl. per-tenant access control); query transforms (multi-query/HyDE) only if eval justifies them.

5. Decide the generation strategy

Grounded prompt + citations + refusal; temperature 0; model tiering (small default, escalate); streaming; output guardrail/faithfulness check.

6. Address the NFRs explicitly with numbers

Latency (cache + rerank-to-few + stream TTFT p90 < 2s), scale (stateless autoscaled service; size the vector DB), cost (token control via routing/caching/tight retrieval), reliability (retries, graceful degradation), security (untrusted-context handling, access filters, PII), compliance (residency, right-to-be-forgotten).

7. Evaluation & operations

Golden-set offline eval in CI; online eval with drift alerts; full tracing; metrics dashboards; feedback loop. State how you'd detect and roll back a regression.

*Throughout: **name the trade-off** for each choice. "I'd use pgvector because we're already on Postgres and at this scale a dedicated DB isn't worth the ops" beats "I'd use Pinecone because it's popular."*

Worked scenario A — Internal documentation assistant

Prompt: "Design a RAG assistant over 10,000 internal documents that change constantly, serving 1,000 concurrent users with cited answers and <2s p90 latency."

1. Clarify: doc types (wiki + PDFs + tickets), update rate (hundreds/day), must cite sources, must respect per-team access permissions, internal-only.

2. Architecture: separated indexing + query pipelines; Qdrant (self-hosted, strong hybrid + filtering); semantic cache; tracing via Langfuse.

3. Indexing: source-specific loaders; recursive chunking ~512/64 (structure-aware for Markdown); text-embedding-3-small; metadata = {source, url, team, updated_at, acl}; **event-driven** re-index on doc change with idempotent upserts + deletes; freshness SLA ~5 min.

4. Retrieval: hybrid (vector + BM25) RRF cross-encoder rerank to 4; **pre-filter by user's team ACL** (security); top-k retrieve 20.

5. Generation: grounded, cited prompt with refusal; gpt-4o-mini default, escalate to gpt-4o for flagged-hard queries; stream tokens.

6. NFRs: <2s p90 via semantic cache (high hit rate on repeated internal questions), rerank-to-few, streaming, warm vector DB; 1,000 concurrent users via horizontally-scaled stateless query service behind a load balancer; cost controlled by caching + mini model + tight top-k; 99.9% uptime via retries + degrade-to-cache.

7. Eval/ops: golden set per team in CI; online eval sampling; trace every request (chunk IDs, scores, prompt, tokens, latency); thumbs feedback grows the golden set; access-control tests in CI.

Likely follow-ups: "How do you stop a user seeing another team's docs?" (ACL pre-filter on every query, tested in CI). "A doc was updated but answers are stale." (check freshness pipeline + cache invalidation).

"Latency spiked." (trace per-stage timings; is it retrieval, rerank, or generation? is the cache cold? did a re-index saturate the DB?).

Worked scenario B — Customer-facing support bot

Prompt: *"Design a public support chatbot over product docs + past tickets. Must be cheap, must not hallucinate, must escalate to a human when unsure."*

Key differences from A: untrusted *public* input (injection/abuse), tighter cost, explicit refusal/escalation, and brand-safety.

Highlights:

- **Guardrails** on input (prompt-injection, abuse, PII) and output (faithfulness check, toxicity); treat retrieved tickets as untrusted (they may contain injected text).
- **Adaptive routing:** chit-chat canned/no-retrieval; FAQ simple RAG; complex multi-step; out-of-scope/low-confidence **escalate to human**.
- **Refusal/escalation** when top rerank score < threshold or faithfulness check fails — never guess to a customer.
- **Cost:** aggressive semantic caching (support questions repeat a lot), mini model, tight top-k.
- **Eval:** golden set from real tickets; track refusal rate and escalation precision; online eval for drift after doc updates.

Likely follow-ups: "How do you prevent a malicious user extracting your system prompt or other customers' data?" (untrusted-context handling + access filters + injection guardrails). "How do you measure 'doesn't hallucinate'?" (faithfulness metric + output guardrail + human review on a sample).

Mock interview (do this on Day 14 — answer out loud, then check yourself)

- 1 Design a RAG system for a law firm to query 2M case documents with citations. What's different about *legal*? (precision, exact citation, access control, domain embeddings, refusal, audit trail)
- 2 Your RAG answers are correct but slow (p90 = 6s). Walk me through diagnosing and fixing it.
- 3 Faithfulness dropped from 0.92 to 0.78 after a "small" prompt change shipped. What happened and how do you prevent it next time?
- 4 The corpus grew from 10k to 10M documents. What breaks and what do you change?
- 5 Build vs buy for a 5-person startup vs a 5,000-person enterprise — argue both.
- 6 A stakeholder demands "100% accuracy." Respond.
- 7 How would you add multi-language support to an existing English RAG system?

8 Walk me through how a single document update propagates to a correct, fresh answer.

Self-scoring rubric (per answer):

- ☐ Clarified requirements before designing.
- ☐ Covered both indexing and query pipelines.
- ☐ Named a metric for the relevant component.
- ☐ Stated at least one explicit trade-off with a reason.
- ☐ Addressed an NFR with a number.
- ☐ Mentioned how you'd evaluate/monitor it.

If you hit 5–6 of those consistently, you're interviewing at architect level.

Phrases that signal seniority

- "Before I design, let me clarify the requirements..."
- "I'd separate indexing from querying so re-indexing never spikes query latency."
- "I'd default to X, but the trade-off is Y; if requirement Z changes I'd switch to W."
- "I'd validate that with the golden set / online eval — I wouldn't ship it unmeasured."
- "The dominant cost here is tokens, so I'd control it with caching, routing, and tight retrieval."
- "I'd treat retrieved content as untrusted to resist prompt injection."